conditional statements.

Why do we need conditional statements?.

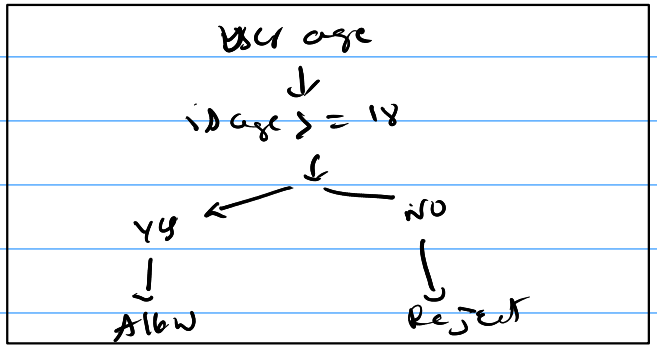
→ condition లేని పక్షంలో, program లోని sequential గా పనిచేస్తుంది.

Example:

```
print("start")
print("check age")
print("you are eligible")
print("end")
```

→ every statement లోని sequential గా execute అవుతుంది
→ But Real world application's Decision మీదనే ఆధారపడి ఉంటుంది.

ex:-



→ This is called control flow.

Three fundamental control flow:

→ ప్రతి program లో మనకు contain అయ్యేవి.

1. Sequence → executes statement one after another
2. Selection → choose which code execute
3. iteration → Repeat the code.

→ Condition statement implements selection / decision making in programi

What is Condition?:

→ Condition అంటే ఏ expression → అది python evaluate చేసి True/false గా return చేస్తుంది.

10>5 → True
10<5 → false. } condition ఇవ్వగానే boolean value ని print చేస్తుంది.

if statement:

→ if అనేది python keyword. if you want a decision-making or selection than we use "if keyword"

Syntax

```
if Condition:
    statement
```

→ if condition True అయితే if block అనేది execute అవుతుంది. లేకపోతే అవదు (false)

Ex:

```
age = 20
if age >= 18:
    print ("adult")
print ("program finished")
```

execution

age → 20
20 >= 18 → True
if True
↓
print "adult"
↙ Then continues
Program finished

What happend when condition False?

```
age = 15
if age >= 18:
    print("adult")
print("program finished")
```

age → 15
↓
15 >= 18
↓
false
↓
if block not executed
↓
Continues
↓
program fished.

The if-else statement :-

→ We often want the two possible paths

```
if Condition:
    statements
else:                    → # else don't have Condition
    statements             meaning Conditions all
                           fall under else:
```

Example :-

```
age = 15
if age >= 18
    print('adult')
else:
    print('minor')
```

flow

age >= 18
/ \
True false
| ↑
adult Minor

→ only one branch executes (this block or that block)
(not both)

The if-elif-else - structures :-

→ if we have the more than two possibility at time we use this

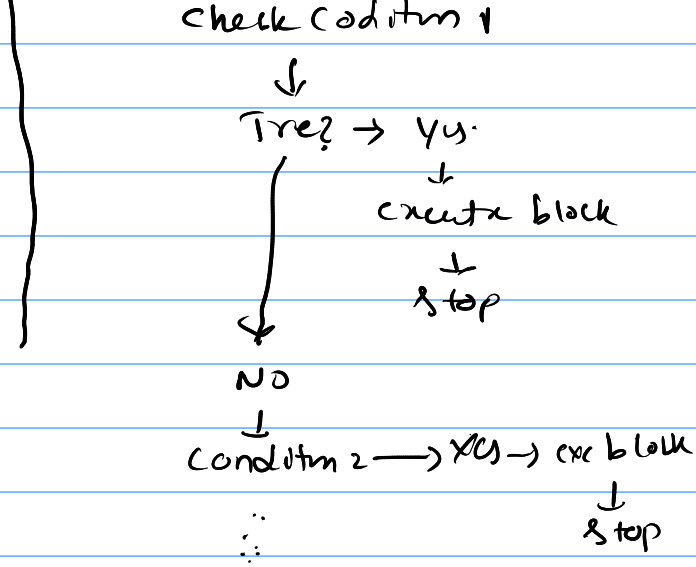
Syntax :-

```
if condition1:
    ---
elif condition2:
    ---
elif condition3:
    ---
else:
    Co.. - -
```

Ex :-

```
marks = 85
if mar >= 90:
    print('A')
elif marks >= 80
    print('B')
elif marks >=70:
    print('C')
else:
    print("f")
```

→ python checks conditions from top to bottom.



if we multiple independent ifs :-

```
x=10
if x>5:
    print('A')
If x>8:
    print('B')
if x>a:
    print('C')
```

A
B
C

} each if statement execute Differently (independently)

→ multiple if → multiple conditions may execute

→ if/elif/else → At most one branch executes.

Nested if :-

→ we can able to one conditional statement another

Ex :-

```
age = 25
has_licence = True
if age >= 18 :
    if has_licence:
        print("you can drive")
```

→ Nested conditions are useful when one depends on another.

Nested if and else :-

```
age = 20
has_licence = True
if age >= 18:
    IF has_licence:
        print("you can drive")
    else:
        print("you need a licence")
else:
    print("you to young")
```

Combination of conditions:

→ python allows you to combine the conditions.

① And ② or ③ NOT

① And → Both conditions are true then and require
→ every think must be true.

Ex:-

```
age = 25
has_licence = True
if age >= 18 and has_licing:
    print("can drive")
```

② Or → at least one condition true

```
day = "Sunday"
if day == "Saturday" or day == "Sunday":
    print("weekend")
```

NOT:

NOT reverse the boolean value.

```
is_logged = false
if not is_logged:
    print("please login")
```

→ a Condition can function call

```
a = "guvi"
if len(a) > 3:
    print(a)  # guvi
```

```
x = 10
y = 20
if x+y > 25:
    print("grater than 25")
```

Short circuit evaluation:-

→ if we are using "AND" it don't need evalute the all the conditions
→ if condition one is false. then it wond check second one.
→ This is called short circuit evaluation.
→ same for OR
→ if the first condition is true then it won't next condition

and and or Don't Necessary Return True/false:-

Ex:- and

```
a = 10
b = 20
res = a and b
print(res) # 20
```

Ex:- or

```
a = 10
b = 20
res = a or b
print(res)    # 10 //
```

Match:-

→ 3.10 way introduced

→ match is using when you're matching structs or patterns

Ex:-

```
Match Command :
    Case "start":
        print ('starting")
    Case 'stop'
        print('stopping')
    Case 'pause"
        Print("pause")
    case '_':
        print ('unknown Comand')
```

Short hand if:-

→ single line statement

```
age = 20
if age>=18: print("adult")
```

short hand if-else:-

```
result = "Adult" if age>=21 else "minior"
```

→ we cannot able to use short hand if-elif-else

→ we can able use nested conditional expressions

```
grade = 'A' if marks>=90 else 'B' if marks>=80 else 'C'
```